Functions and Control Flow

DATA 118 Lab — Student Version

Goals

- To become familiar working with basic functions and writing our own.
- To explore using functions as arguments to other functions.
- To implement control flow in our code, which alters the linear path of code execution through:
  - Conditional statements: `if, elif, else` blocks
  - Iteration: `for` and `while` loops

Introduction and motivation

Up until this point, we have been writing code that executes linearly: Python reads our code line by line, without skipping or repeating any.

In this lab, we explore how to write Python code that repeats certain blocks, or skips them when they are not necessary. You might be wondering why we might want to do this. Consider the following scenarios:

- I am a researcher who runs experiments and calculates field-specific summary statistics at the end of each experiment. I don't want to repeat the code for my statistic calculation every time an experiment ends.
- I am writing code to automate sending birthday texts. However, I only want this code to execute **if** I haven't already sent a birthday text that day.
- I would like to print every element of a (very long) list, each on a new line, without explicitly writing code for each print statement.

All of these problems can be solved either with functions, control statements, or a combination of the two!

Beyond saving our own time and sanity, writing code that uses functions and control statements (conditionals and iteration) is a good organizational practice. One important facet of writing code is to consider its complexity – we want our code to be easily understood by anyone who looks over it. Functions and control statements, alongside other good practices like informative variable naming and documenting our code with comments, help us produce code that is easy to hand off to another person (or our future selves).

Setup

Consider the following example on movie theater finances:

- Profit: defined as revenue (money from ticket sales) – cost (studio fees)
- Revenue: calculated from ticket price multiplied by the number of attendees
- Cost: $\$180$ studio fee plus four cents for every attendee

Mathematically, this means:

$$\text{profit} = \text{price} \times (\text{attendees}) - (180 + 0.04 \times \text{attendees})$$

So, if tickets cost \$5 and we have 120 attendees, we can calculate the profit. Translate the information above into Python code in the following chunk:

Exercise

```
revenue = ...
cost = ...
profit = ...
profit
```

Constants and variables

Notice that the information from our motivating example depends on many variables. Some will change with time, and other won't.

First let's consider our constants: the studio fee and cost per attendee. Even though these values are likely to stay fixed over time, it is good practice to define them using variables in our code. This is both for readibility and ease of change.

Check out the Python style guide that we use in DATA120. According to this guide, it is best practice to write named constants in all capital letters at the top of our file, so it's easy to find them and understand what they are when reading code. Also, we prefer to write decimal values with leading zeros: `0.8` is preferred to `.8`.

Let's do so below:

Exercise

```
STUDIO_FEE =
COST_PER_ATTENDEE =
```

On the other hand, ticket price and number of attendees are dynamic, and likely to change over time. Therefore, they are better suited as *inputs* to our code chunk. At this point, we should consider moving our code into a **function**. Let's do that in the next section.

Function basics

Because we have a code chunk that we'd like to repeat with different values, our profit calculation is a great use case for a function.

Let's rewrite our profit calculation as a function called `calculate_profit` that takes arguments `ticket_price` and `num_attendees`. Write your constants above the function.

Exercise

```
# CONSTANTS

# YOUR FUNCTION IS DEFINED BELOW

......................
    revenue = ticket_price * num_attendees
    cost = STUDIO_FEE + COST_PER_ATTENDEE * num_attendees
    profit = revenue - cost
    ........
```

Check to see that your function works as expected by calling it with `ticket_price = 5` and `num_attendees = 120` again.

Exercise — write your code here:

Assume that we know that for every `$1` discount, `40` more people will come. What is the profit if we set `ticket_price = 4`? Use your function to figure out the answer.

Exercise — write your code here:

Question 1. What is our new profit if we decrease the ticket price to $4?
☐ $415.20
☐ $295.20
☐ $619.80
☐ $453.60

Great! Now we have a working function that allows us to calculate the new profit as ticket price and number of attendees changes.

Docstrings

Remember that we should be documenting what our functions do using docstrings.

Question 2. Which of the following are true about docstrings? *(Select all that apply.)*
☐ We can access them later using `.__doc__`
☐ We can access them later using `.__docstring__`
☐ We enclose docstrings in triple quotes
☐ Docstrings should be placed above the `def func()` line
☐ Docstrings should be placed below the `def func()` line, and indented with the function body

Now go back and add a docstring to your function if it doesn't have one!

Beyond basic functions

Before we begin making our functions more complex, let's cover a few more function-related topics that will come in handy.

`lambda` functions

We might find it unnecessary to write entire functions for very short computations. For example, revenue is just ticket price times attendees. Writing out a full `def` block for this feels a little clunky:

```
def calculate_revenue(ticket_price, num_attendees):
    return ticket_price * num_attendees
```

Python lets us write small, one-line functions using the `lambda` keyword. The general syntax is: `lambda arguments: expression`.

Our arguments go before the colon, and the single expression after the colon is what the function returns. Notice that there is no function name or `return` keyword; the result of the expression is returned automatically. Here is the same revenue calculation written as a `lambda` function:

```
calculate_revenue = lambda ticket_price, num_attendees: ticket_price *
 ↪  num_attendees
```

Because `lambda` functions don't *need* a name, they are sometimes called **anonymous functions**. We assigned this one to the variable `calculate_revenue` so we could call it, but `lambda` functions are actually most useful when we pass them directly into another function without ever naming them. We'll see this in the next section.

Now, write a `lambda` function that you assign to a variable `calculate_cost` that takes `num_attendees` and returns the total cost: the studio fee plus the cost per attendee times the number of attendees. Use the named constants `STUDIO_FEE` and `COST_PER_ATTENDEE`.

Exercise

```
calculate_cost = ...
```

Recall that we assumed every `$1` discount from our `$5` ticket price brings in `40` more people, starting from `120` attendees. Write a `lambda` function called `estimate_attendees` that takes `ticket_price` and returns the expected number of attendees.

Exercise

```
estimate_attendees = lambda ticket_price: ...

estimate_attendees(4)
```

Question 3. Which of the following are true about `lambda` functions? *(Select all that apply.)*

☐ The value of the expression after the colon is returned automatically
☐ They must include the `return` keyword
☐ They can contain multiple lines of statements, just like a `def` function
☐ They must be assigned to a variable name before they can be used
☐ They can take more than one argument

Functions as objects

In Python, *everything* is an object: integers, strings, lists, data frames… and functions too! This means a function can be treated like any other value. We can check its type, assign it to a new variable, or store it in a list.

```
print(type(calculate_profit))

profit_function = calculate_profit
print(profit_function(5, 120))
```

```
<class 'function'>
415.2
```

Notice that when we wrote `profit_function = calculate_profit`, we did **not** use parentheses. There is an important difference here:

- `calculate_profit` refers to the function object itself
- `calculate_profit(5, 120)` *calls* the function, running the code and giving back the result

Question 4. What is stored in `x` after running `x = calculate_profit`?
☐ The number `415.2`
☐ The function `calculate_profit` itself, which we can call later with `x(5, 120)`
☐ Nothing, because this line raises an error since no arguments were given
☐ The string `'calculate_profit'`

Passing functions as arguments Because functions are objects, we can pass them as arguments to other functions.

For example, our theater manager might have different models for how many people will show up at a given price. Instead of hard-coding one model, which is inflexible, we can write a function that takes `attendance_func` as an argument:

```
def profit_at_price(ticket_price, attendance_func):
    """
    Calculate the profit at a given ticket price, using attendance_func
    to estimate the number of attendees at that price.
    """
    num_attendees = attendance_func(ticket_price)
    return calculate_profit(ticket_price, num_attendees)
```

Try running calling `profit_at_price` with `ticket_price = 4`, and use our function `estimate_attendees` as `attendance_func`.

Exercise — write your code here:

Now, let's say that on weekends, the theater is busier: `200` people come at `$5`, and each `$1` discount brings in `50` more. We don't need to write a new named function for this. We can pass a `lambda` directly:

```
profit_at_price(4, lambda ticket_price: 200 + 50 * (5 - ticket_price))
```

```
810.0
```

Discuss with a neighbor (or on Ed):
Many built-in Python functions also accept functions as arguments. For example, `max()` has an optional `key` argument: a function used to score each element. As a challenge, try to write a line of code to find which ticket price in a list will give the highest profit given our weekday model of estimating attendance.

Returning functions Just as functions can be passed *into* other functions, they can also be returned *from* functions. This lets us write a function that builds and hands back new, customized functions.

Earlier we wrote two attendance models by hand, one for weekdays and one for weekends. They have the same shape and only differ in their numbers. Instead, we can write a function that *creates* attendance models:

```
def make_attendance_model(base_attendees, base_price,
↪  attendees_per_dollar):
    """
    Return a function that estimates attendance at a given ticket price.
    """
    return lambda ticket_price: base_attendees + attendees_per_dollar *
    ↪  (base_price - ticket_price)
```

Now play around a bit with `make_attendance_model`. Try to use it to recreate the weekday and weekend models we used earlier.

Exercise — write your code here:

Question 5. True or false: Running `weekday_model = make_attendance_model(120, 5, 40)` and `weekend_model = make_attendance_model(200, 5, 50)` calculates the expected attendance on weekdays and weekends.
☐ True
☐ False

Conditional statements

As you've gone about creating functions, you may have noticed times where it would have been useful to introduce some logic to your code. To put it in words: "if x is true,

then execute *this* code, but if y is true, execute *that* code". To implement such logic, we use **conditional statements**.

As a quick refresher, conditional statements in Python look like this:

```
if condition:
    # runs if condition is True
elif other_condition:
    # runs if condition is False and other_condition is True
else:
    # runs if none of the conditions above are True
```

Python checks each condition from top to bottom and runs only the **first** block whose condition is `True`. The `elif` and `else` parts are optional, and we can have as many `elif` blocks as we like. Conditions are usually built from comparisons (`==`, `!=`, `<`, `>`, `<=`, `>=`), membership checks (`in`), and logical operators (`and`, `or`, `not`).

Wednesday discount day

Let's return to our theater. To bring in more customers midweek, the theater runs a special: **every Wednesday, tickets are only `$3`**. Weekends are busy, so tickets on Saturday and Sunday cost `$6`. Every other day, tickets are the regular `$5`. On top of that, weekends use our busier `weekend_model` for attendance, while weekdays use the `weekday_model`.

We'd like a function `daily_profit(day)` that answers questions like "if today is Wednesday, what is the ticket price, and how much profit do we make?"

Trying to write this all at once, from scratch, would be messy: we'd be juggling prices, attendance models, and the profit formula inside one tangled function. When writing functions, aim for bite size pieces! Writing a very large program is an intimidating and daunting task, but if you break it up into smaller pieces and figure out how to solve each of those pieces separately, it is much easier.

So let's break `daily_profit` into smaller questions:

1. What is the ticket price on a given day?
2. Which attendance model should we use on a given day?
3. Given those two pieces, what is the profit? (We already have `profit_at_price` for this!)

Step 1: ticket price First, let's write `get_ticket_price(day)`, which returns the ticket price for a given day. We've written the constants for the prices and the weekend days for you. It's your job to fill in the conditions and return values.

Exercise

```
DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
 ↪  "Saturday", "Sunday"]
WEEKEND = ["Saturday", "Sunday"]
REGULAR_PRICE = 5
WEDNESDAY_PRICE = 3
WEEKEND_PRICE = 6

def get_ticket_price(day):
    if day == ...:
        return ...
    elif ...:
        return ...
    else:
        return ...
```

Hint: For the `elif` condition, use `in` to check whether `day` is one of the days in the `WEEKEND` list. For example, `"Sunday" in ["Saturday", "Sunday"]` is `True`.

Question 6. Answer this question without running any code: what will `get_ticket_price("wednesday")` return?

☐ 3
☐ 5
☐ 6
☐ An error

You might be tempted to write a series of `if` statements instead of `elif` and `else`, but this is often bad practice. Let's see why below:

```
def get_ticket_price(day):
    if day == "Wednesday":
        ticket_price = WEDNESDAY_PRICE
    if day in WEEKEND:
        ticket_price = WEEKEND_PRICE
    else:
        ticket_price = REGULAR_PRICE
    return ticket_price
```

Question 7. Answer this question without running any code: what will `get_ticket_price("Wednesday")` return now?

☐ 3
☐ 5
☐ 6
☐ An error, because `ticket_price` is assigned more than once

Step 2: attendance model Next, write `get_attendance_model(day)`. It should return `weekend_model` on weekends and `weekday_model` otherwise. Notice that this function returns a *function*, just like we saw in the last section!

Exercise

```
def get_attendance_model(day):
    if ...:
        return ...
    return ...
```

Note that this code only works if we created the weekend and weekday models earlier!

Discuss with a neighbor (or on Ed):
We wrote our code chunk above without an `else` statement. Was that a mistake? Do we need one here?

Step 3: putting it together Now that each piece works on its own, building `daily_profit(day)` is much easier. Use `get_ticket_price`, `get_attendance_model`, and `profit_at_price` to complete the function.

Exercise

```
def daily_profit(day):
    """
    Calculate the theater's profit on a given day of the week.
    """
    ticket_price = ...
    attendance_func = ...
    return ...

daily_profit("Wednesday")
```

Notice how short and readable `daily_profit` is. Each line reads almost like a sentence, and if something goes wrong, we can test each smaller function on its own to find the problem.

Use the chunk below to calculate the profit on a few different days.

Exercise — write your code here:

Question 8. How much profit does the theater make on a Wednesday? How about on a Monday?
☐ $412.00 on Wednesday, $415.20 on Monday
☐ $453.60 on Wednesday, $415.20 on Monday
☐ $412.00 on Wednesday, $714.00 on Monday
☐ $600.00 on Wednesday, $600.00 on Monday

Iteration

Using conditional statements, we can calculate the profit for any single day. But what if the theater manager asks, "how much would we make in a **week**?" We could call `daily_profit` seven times and add the results by hand, but that's repetitive, error-prone, and won't scale if we're asked about a month or a year. Instead, we can use a **loop** to repeat code for us.

`for` loops

A `for` loop runs a block of code once for each element in a sequence:

```
for item in sequence:
    # this code runs once for each item
```

Each time we go through the loop, `item` is assigned the next value in `sequence`. Once we reach the end of `sequence`, Python moves on to the code after the loop.

A very common sequence to loop over is a range of numbers created with `np.arange()`. Recall that `np.arange(start, stop, step)` creates an array of numbers starting at `start`, going up by `step`, and stopping *before* `stop`.

```
print(np.arange(7))
print(np.arange(3, 6.5, 0.5))
```

```
[0 1 2 3 4 5 6]
[3.  3.5 4.  4.5 5.  5.5 6. ]
```

Since `np.arange(7)` gives us `0` through `6`, we can use these numbers as indices into our `DAYS` list. Here is a loop that prints the profit for each day of the week:

```
for i in np.arange(7):
    day = DAYS[i]
    print(day, daily_profit(day))
```

```
Monday 415.2
Tuesday 415.2
Wednesday 415.2
Thursday 415.2
Friday 415.2
Saturday 714.0
Sunday 714.0
```

Adding up a week of profit

Printing is nice, but we want a single total. To do this, we create a variable *before* the loop that starts at `0`, and add to it each time through the loop.

Complete the loop below so that `total` holds the theater's total profit for one week.

Exercise

```
total = 0

for i in np.arange(...):
    day = ...
    total = ...

total
```

Question 9. What would happen if we moved `total = 0` *inside* the body of the loop?
☐ Nothing, the result would be the same
☐ `total` would be reset to `0` every time through the loop, so it would only hold Sunday's profit at the end
☐ Python would raise an error
☐ `total` would be seven times larger

Looping with `enumerate()` In the loop above, we used `np.arange(7)` to get an index `i`, and then used `DAYS[i]` to look up the day. This works, but Python gives us a cleaner option: `enumerate()`. When we loop over `enumerate(DAYS)`, each step gives us *two* values at once: the index and the element at that index.

```
total = 0

for i, day in enumerate(DAYS):
    profit = daily_profit(day)
    total = total + profit
    print("Day", i + 1, "is", day, "with profit", profit)

total
```

```
Day 1 is Monday with profit 415.2
Day 2 is Tuesday with profit 415.2
Day 3 is Wednesday with profit 415.2
Day 4 is Thursday with profit 415.2
Day 5 is Friday with profit 415.2
Day 6 is Saturday with profit 714.0
Day 7 is Sunday with profit 714.0
3504.0
```

Notice that we no longer need `DAYS[i]` or to know that there are exactly 7 days; `enumerate()` handles both for us. The index `i` still starts at `0`, which is why we print `i + 1`. If we didn't need the index at all, we could simply write `for day in DAYS:`.

A quick note on `while` loops

A `for` loop is the right tool when we know ahead of time how many times to repeat (for example, 7 days). Sometimes, though, we want to keep repeating *until a condition is met*, and we don't know in advance how long that will take. For this, Python has `while` loops:

```
while condition:
    # this code runs again and again, as long as condition is True
```

For example, suppose the theater wants to save up `$10,000` for a new projector. How many days will that take?

```
PROJECTOR_COST = 10000

savings = 0
num_days = 0

while savings < PROJECTOR_COST:
    day = DAYS[num_days % 7]
    savings = savings + daily_profit(day)
    num_days = num_days + 1

print(num_days, savings)
```

```
21 10512.0
```

Be careful: the body of a `while` loop must eventually make the condition `False`. If we forgot the line `savings = savings + daily_profit(day)`, `savings` would stay at `0` forever, and the loop would never stop. This is called an **infinite loop**. In practice, you'll use `for` loops much more often, but `while` loops are handy when the number of repetitions isn't known ahead of time.